DOCUMENT EXÉCUTIF — INTELLIGENCE ARTIFICIELLE & SYSTÈMES MULTI-AGENTS

Autonomous Research Agent

ARA — Version 2.0 | Documentation Technique Pédagogique Complète

Un système multi-agents basé sur l'Intelligence Artificielle qui automatise entièrement la veille scientifique et la rédaction de rapports académiques.

Documentation complète — Fonctions Python ligne par ligne

Thème de recherche actuel :

"Comment concevoir des politiques de décarbonation et des structures fiscales qui garantissent l'équité intergénérationnelle sans aggraver les inégalités ?"

Responsable : Bilel Ammouri

Date : Avril 2025

Volume : ≈60 pages | 6 agents | 18 fonctions documentées

Classification : Document Exécutif Confidentiel

Version étendue avec documentation complète des fonctions Python (Input / Body / Output).

Table des matières

Résumé Exécutif 3

1 Contexte et Problématique 4
- 1.1 Le défi de la veille scientifique moderne . . . 4
- 1.2 Les coûts de la veille manuelle . . . 4
- 1.3 La solution multi-agents . . . 4

2 Architecture Générale du Système 5
- 2.1 Vue d'ensemble — Le pipeline complet . . . 5
- 2.2 Résumé des échanges de données . . . 6

3 Carte Mentale Globale du Système 7

4 Agent 1 — Master Agent (`master_agent.py`) 8
- 4.1 Rôle et positionnement . . . 8
- 4.2 Fonction 1.1 — `find_r_executable()` . . . 8
- 4.3 Fonction 1.2 — `run_step()` . . . 9
- 4.4 Fonction 1.3 — `merge_results()` . . . 11
- 4.5 Fonction 1.4 — `main()` . . . 13

5 Agent 2 — Scopus Collector (`core/scopus_collector.py`) 16
- 5.1 Rôle et positionnement . . . 16
- 5.2 Fonction 2.1 — `get_config()` . . . 16
- 5.3 Fonction 2.2 — `suggest_boolean_query()` . . . 17
- 5.4 Fonction 2.3 — `fetch_scopus_data()` . . . 19
- 5.5 Fonction 2.4 — `process_entries()` . . . 21
- 5.6 Fonction 2.5 — `export_to_bibtex()` (version Scopus) . . . 23

6 Agent 3 — OpenAlex Collector (`core/openalex_collector.py`) 25
- 6.1 Rôle et positionnement . . . 25
- 6.2 Fonction 3.1 — `reconstruct_abstract()` . . . 25
- 6.3 Fonction 3.2 — `suggest_search_query()` (version OpenAlex) . . . 27
- 6.4 Fonction 3.3 — `fetch_openalex_data()` . . . 28
- 6.5 Fonction 3.4 — `process_results()` . . . 29

7 Agent 4 — Analysis Engine R (`core/analysis_engine.R`) 32
- 7.1 Rôle et positionnement . . . 32
- 7.2 Analyse du script R complet . . . 32

8 Agent 5 — Paper Ranker (`core/paper_ranker.py`) 34
- 8.1 Rôle et positionnement . . . 34
- 8.2 Fonction unique — `rank_papers()` . . . 34

9 Agent 6 — Review Drafter (`core/review_drafter.py`) 38
9.1 Rôle et positionnement . . . 38
9.2 Fonction 6.1 — `load_config()` et `load_data()` . . . 38
9.3 Fonction 6.2 — `latex_escape()` . . . 39
9.4 Fonction 6.3 — `initialize_client()` . . . 40
9.5 Fonction 6.4 — `call_with_retry()` . . . 41
9.6 Fonction 6.5 — `generate_dynamic_structure()` . . . 42
9.7 Fonction 6.6 — `draft_section()` . . . 44
9.8 Fonction 6.7 — `create_latex_master()` . . . 46
9.9 Fonction 6.8 — `run_compilation()` . . . 47

10 Configuration Complète du Système 49
10.1 Le fichier `agent_config.json` — Tableau de Bord Unique . . . 49
10.2 Modifier le thème de recherche . . . 50

11 Approche Pédagogique : Du Simple au Complexe 51
11.1 Les trois niveaux de compréhension . . . 51
11.2 Tableau comparatif des analogies utilisées . . . 51

12 Tableau de Bord Complet des Fonctions 53

13 Résilience et Gestion des Erreurs 54
13.1 Stratégie de repli (Fallback Strategy) . . . 54
13.2 Flux de résilience visualisé . . . 54

14 Guide d'Utilisation Pratique 55
14.1 Installation et prérequis . . . 55
14.2 Installation des dépendances Python . . . 55
14.3 Lancement du pipeline complet . . . 55
14.4 Lancement d'un agent individuel (débogage) . . . 55
14.5 Arborescence des fichiers générés . . . 56

15 Conclusion et Recommandations Stratégiques 57
15.1 Pistes d'évolution recommandées . . . 57

Résumé Exécutif

En 30 secondes : de quoi s'agit-il ?

ARA est un système informatique qui **travaille comme une équipe de chercheurs autonomes**. Vous lui donnez un sujet de recherche. En retour, il interroge les plus grandes bases de données scientifiques mondiales, évalue chaque article trouvé, sélectionne les meilleurs, puis rédige un rapport académique de qualité professionnelle — **sans aucune intervention humaine**.

Dimension	Description
Problème adressé	La veille scientifique manuelle prend des semaines et mobilise des experts coûteux
Solution proposée	Un pipeline automatisé multi-agents qui effectue ce travail en quelques minutes
Bénéfice principal	Gain de temps $\times 100$, reproductible à l'infini, coût marginal quasi nul
Sources de données	Scopus (Elsevier) + OpenAlex (250M articles en libre accès)
Technologie IA	Gemini (Google DeepMind) + R Bibliometrix
Résultat livrable	Rapport PDF académique + données structurées CSV/JSON/BibTeX
Agents	6 agents spécialisés, 18 fonctions Python documentées

Ce qu'il faut retenir

- ARA = 6 programmes spécialisés (agents) travaillant en séquence
- Chaque agent fait une chose précise et passe le relais au suivant
- Le résultat final est un rapport PDF académique de qualité publication
- Toute la chaîne se contrôle via un seul fichier de configuration

1. ◎ Contexte et Problématique

1.1. Le défi de la veille scientifique moderne

La production scientifique mondiale croît de façon exponentielle. En 2023, plus de **3 millions de nouveaux articles** ont été publiés dans des revues indexées. Face à ce déluge d'information, les acteurs académiques, institutionnels et décisionnels font face à un défi structurel.

Analogie : comprendre facilement

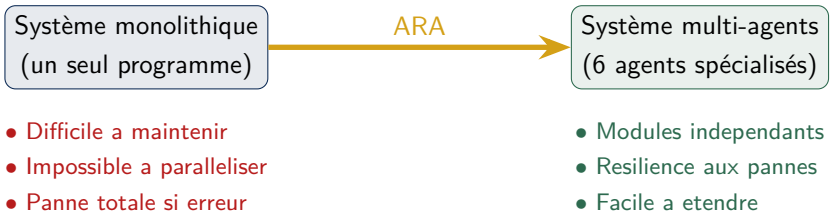
Imaginez une bibliothèque qui reçoit chaque jour l'équivalent de **8 000 nouveaux livres**. Un bibliothécaire seul ne peut pas tout lire, classer et résumer. C'est exactement le problème que résout ARA : il est le bibliothécaire ultra-rapide qui peut tout traiter en quelques minutes.

1.2. Les coûts de la veille manuelle

Tâche	Sans ARA	Avec ARA	Gain
Formulation requête	2–4 heures	30 secondes	$\times 480$
Collecte d'articles	1–3 jours	5 minutes	$\times 860$
Tri et évaluation	3–5 jours	2 minutes	$\times 2160$
Rédaction synthèse	2–4 semaines	8 minutes	$\times 5040$
Total	**3–6 semaines**	**15 minutes**	$>\times$**1000**

1.3. La solution multi-agents

Un *système multi-agents* est une architecture logicielle où plusieurs programmes spécialisés (les "agents") collaborent pour résoudre un problème complexe. Chaque agent est expert dans un domaine étroit, ce qui permet une **spécialisation**, une **parallélisation** et une **résilience** que des systèmes monolithiques ne peuvent pas atteindre.



2. Architecture Générale du Système

2.1. Vue d'ensemble — Le pipeline complet

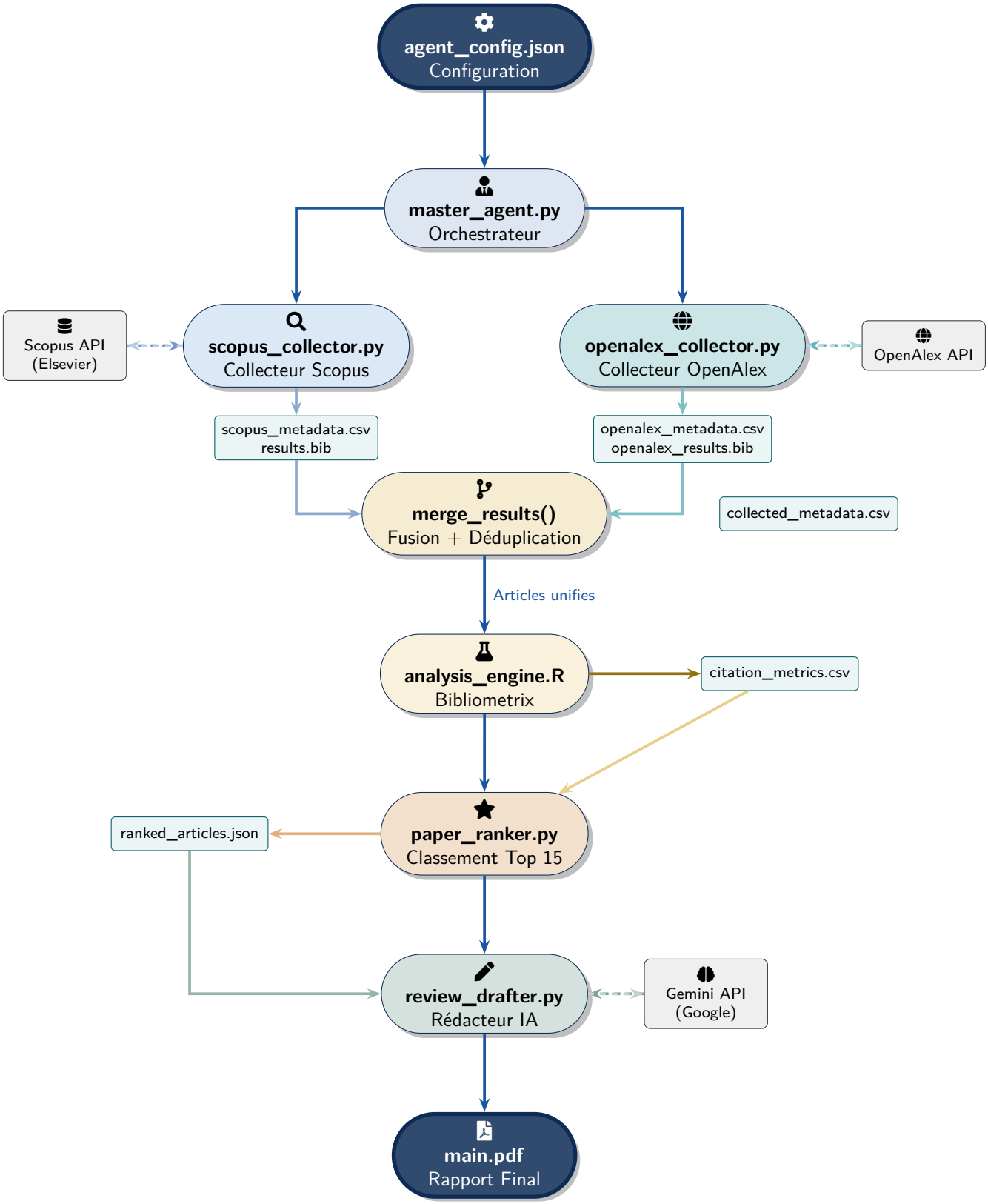


FIGURE 1 – Architecture complete du pipeline ARA – flux de donnees et d'appels entre agents

2.2. Résumé des échanges de données

Agent	Fichier d'entrée	Fichier de sortie	Format
master_agent.py	agent_config.json	— (orchestrateur)	JSON
scopus_collector.py	agent_config.json	scopus_metadata.csv, results.bib	CSV, BibTeX
openalex_collector.py	agent_config.json	openalex_metadata.csv, openalex_results.bib	CSV, BibTeX
merge_results()	*_metadata.csv, *.bib	collected_metadata.csv, results.bib	CSV, BibTeX
analysis_engine.R	results.bib	citation_metrics.csv, bibliometric_summary.png	CSV, PNG
paper_ranker.py	collected_metadata.csv	ranked_articles.json	JSON
review_drafter.py	ranked_articles.json	main.tex, main.pdf	LaTeX, PDF

3. Carte Mentale Globale du Système

FIGURE 2 – Mindmap globale – vue d'ensemble des 6 agents et leurs fonctions

4. Agent 1 — Master Agent (`master_agent.py`)

4.1. Rôle et positionnement

Master Agent — Le Chef d'Orchestre

Rôle métier	Coordonner l'ensemble du pipeline de bout en bout
Analogie	Un chef de projet qui reçoit la commande client, distribue le travail, surveille l'avancement et consolide la livraison
Entrée	`agent_config.json` — fichier de configuration
Sortie	Déclenchement ordonné de tous les agents
Technologie	Python 3.10+ — module `subprocess` pour appels système

4.2. Fonction 1.1 — `find_r_executable()`

SIGNATURE COMPLÈTE — `find_r_executable() → str | None`

Localise automatiquement l'exécutable Rscript sur le système, qu'il soit dans le PATH ou dans les chemins d'installation standard de Windows.

INPUTS — Données d'entrée

Aucun paramètre d'entrée (la fonction explore elle-même le système).
La fonction cherche dans deux endroits :

- Le *PATH système* (variable d'environnement globale qui liste les dossiers où Windows cherche les exécutables)
- Une liste de chemins d'installation standard de R sur Windows

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def find_r_executable():
    # Etape 1 : chercher Rscript dans le PATH systeme
    if shutil.which("Rscript"):     # shutil.which = equivalent de
        return "Rscript"            # "where Rscript" en PowerShell

    # Etape 2 : liste de chemins Windows courants
    common_paths = [
        r"C:\Program Files\R\R-4.2.2\bin\Rscript.exe",
        r"C:\Program Files\R\R-4.3.0\bin\Rscript.exe",
        r"C:\PROGRA~1\R\R-4.2.2\bin\Rscript.exe"  # chemin court
    ]

```

```
13      # Etape 3 : tester chaque chemin
14      for path in common_paths:
15          if os.path.exists(path):    # est-ce que ce fichier existe ?
16              return path             # oui -> retourner ce chemin
17
18      return None  # R introuvable sur ce systeme
```

Listing 1 – Corps de find_r-executable()

Explication ligne par ligne :

1. **L.2** : `shutil.which()` fait l'équivalent de `where Rscript` en PowerShell — si R est dans le PATH, retourne immédiatement son chemin
2. **L.5–9** : Si R n'est pas dans le PATH, on teste manuellement les dossiers d'installation les plus courants sur Windows
3. **L.11–13** : Pour chaque chemin dans la liste, `os.path.exists()` vérifie si le fichier existe physiquement sur le disque
4. **L.15** : Si aucun chemin ne fonctionne, retourner `None` (valeur Python signifiant "rien trouvé")

→ OUTPUT — Résultat produit

— `str` : Le chemin complet vers l'exécutable Rscript (ex : `"C:\Program Files\R\R-4.3.0\bin\Rscript.exe"`)

— `None` : Si R n'est pas installé ni trouvable sur le système

Cette valeur sera utilisée par `run_step()` pour remplacer la commande générique "Rscript" par le chemin absolu réel.

4.3. Fonction 1.2 — `run_step()`

</> SIGNATURE COMPLÈTE — `run_step(name: str, command: list) → bool`

Exécute une étape du pipeline sous forme de commande système, avec gestion des erreurs.

INPUTS — Données d'entrée

Paramètre	Type	Description
name	str	Nom lisible de l'étape (ex : "Scopus Data Collection") — utilisé uniquement pour l'affichage
command	list	Commande à exécuter sous forme de liste (ex : ["python", "core/scopus_collector.py"])

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def run_step(name, command):
    # Affichage decoratif dans la console
    print(f"\n" + "="*50)
    print(f"  STEP: {name}")
    print("="*50)

    # Si la commande utilise Rscript, trouver le vrai chemin
    if command[0] == "Rscript":
        r_path = find_r_executable()    # appel a la fct precedente
        if r_path:
            command[0] = r_path         # remplace "Rscript" par chemin
        else:
            print("[ERROR] Rscript not found.")
            return False  # echec = retourner False

    try:
        # subprocess.run = executer une commande systeme
        result = subprocess.run(command, check=True)
        # check=True leve une exception si code retour != 0
        return True  # succes
    except subprocess.CalledProcessError:  # commande echouee
        print(f"[ERROR] Step '{name}' failed.")
        return False
    except FileNotFoundError:  # executable introuvable
        print(f"[ERROR] Executable not found for: {name}")
        return False
```

Listing 2 – Corps de run-step()

Concepts clés expliqués :

Concept : subprocess.run()

C'est la façon dont un programme Python "appelle" un autre programme externe (comme si vous tapiez une commande dans PowerShell). Le paramètre `check=True` fait que si le programme externe retourne une erreur, une exception est levée automatiquement.

Concept : try / except

Mécanisme de gestion d'erreurs en Python. Si quelque chose se passe mal à l'intérieur du `try`, Python "attrape" l'erreur dans le `except` au lieu de planter. Ici, deux types d'erreurs sont gérés : l'échec de la commande et l'absence de l'exécutable.

→ OUTPUT — Résultat produit

— `True` : L'étape s'est exécutée avec succès (code retour 0)

— `False` : L'étape a échoué (une des exceptions a été levée)

La valeur de retour est utilisée par `main()` pour décider si le pipeline doit continuer ou s'arrêter.

4.4. Fonction 1.3 — `merge_results()`

SIGNATURE COMPLÈTE — `merge_results(sources: list) → None`

Fusionne les données collectées depuis Scopus et/ou OpenAlex, déduplique les articles, et sauvegarde le fichier unifié.

INPUTS — Données d'entrée

Paramètre	Type	Description
`sources`	`list[str]`	Liste des sources activées, ex : `["scopus", "openalex"]` ou `["scopus"]` uniquement

Les fichiers CSV et BibTeX présents sur le disque sont les vraies "données d'entrée" :

— `scopus_metadata.csv` si `"scopus"` est dans sources

— `openalex_metadata.csv` si `"openalex"` est dans sources

— `results.bib` et `openalex_results.bib`

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```
def merge_results(sources):
    import pandas as pd      # bibliotheque pour manipuler des
```

```
        tableaux
    dfs = []                    # liste vide de DataFrames
    bib_contents = []           # liste vide de contenus BibTeX

    # 1. Charger les donnees Scopus si disponibles
    if "scopus" in sources:
        if os.path.exists("scopus_metadata.csv"):
            dfs.append(pd.read_csv("scopus_metadata.csv"))
        if os.path.exists("results.bib"):
            with open("results.bib", "r", encoding="utf-8") as f:
                bib_contents.append(f.read())

    # 2. Charger les donnees OpenAlex si disponibles
    if "openalex" in sources:
        if os.path.exists("openalex_metadata.csv"):
            dfs.append(pd.read_csv("openalex_metadata.csv"))
        if os.path.exists("openalex_results.bib"):
            with open("openalex_results.bib", "r", encoding="utf-8") as f:
                bib_contents.append(f.read())

    # 3. Fusionner et dedupliquer les CSV
    if dfs:
        combined_df = pd.concat(dfs, ignore_index=True)
        # ignore_index : reindexer de 0 a N apres concatenation

        # Deduplication par DOI (identifiant unique d'un article)
        if "prism:doi" in combined_df.columns:
            combined_df["temp_doi"] = combined_df["prism:doi"].replace("", None)
            combined_df = combined_df.drop_duplicates(
                subset=["temp_doi"], keep="first"
            ).drop(columns=["temp_doi"])

        # Deduplication par titre si DOI absent
        combined_df = combined_df.drop_duplicates(
            subset=["dc:title"], keep="first"
        )
        combined_df.to_csv("collected_metadata.csv", index=False)
        print(f"[Orchestrator] Merged: {len(combined_df)} unique articles.")

```

```
    # 4. Fusionner les fichiers BibTeX
    if bib_contents:
        with open("results.bib", "w", encoding="utf-8") as f:
            f.write("\n\n".join(bib_contents))
```

Listing 3 – Corps de merge-results()

Le processus de déduplication expliqué simplement :

Analogie : comprendre facilement

Imaginez que vous avez deux listes d'invités pour un événement — une de Scopus et une d'OpenAlex. Certaines personnes apparaissent sur les deux listes. La déduplication consiste à fusionner les listes en supprimant les doublons.
Deux critères sont utilisés, dans l'ordre :

1. **Le DOI** (Digital Object Identifier) : identifiant unique universel d'un article — c'est comme le numéro de passeport d'un article
2. **Le Titre** : si le DOI est absent, on compare les titres pour éviter les doublons

OUTPUT — Résultat produit

Aucune valeur de retour (`None`). Les effets sont :

— Création de `collected_metadata.csv` — tableau unifié sans doublons

— Mise à jour de `results.bib` — fichier de bibliographie fusionné

— Affichage du nombre d'articles uniques retenus

4.5. Fonction 1.4 — `main()`

SIGNATURE COMPLÈTE — `main()` → `None`

Point d'entrée principal du programme. Orchestre l'ensemble du pipeline en séquence.

INPUTS — Données d'entrée

Aucun paramètre direct. La fonction lit elle-même `agent_config.json` et sollicite une entrée interactive de l'utilisateur (choix de la source).

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```
def main():
    # 0. Verification de la cle API Gemini
    import json
    if os.path.exists("agent_config.json"):
        with open("agent_config.json", "r", encoding="utf-8")
```

```
            as f:
            config = json.load(f)
            if config.get("gemini_api_key") == "YOUR_GEMINI_API_KEY_HERE":
                print("\n" + "!"*50)
                print("  [ACTION REQUIRED] AJOUTEZ VOTRE CLE API GEMINI")
                print("!"*50 + "\n")
                sys.exit(1)     # arreter le programme avec code erreur 1

    # 1. Menu de selection de la source
    choice = input("\n  Select source [1-3, default=3]: ").strip()
    if choice == "1":
        sources = ["scopus"]
    elif choice == "2":
        sources = ["openalex"]
    else:
        sources = ["scopus", "openalex"]  # par defaut : les deux

    # 2. Collecte Scopus (si selectionnee)
    if "scopus" in sources:
        if not run_step("Scopus Data Collection",
                        ["python", "core/scopus_collector.py"]):
            print("[Warn] Scopus collection failed.")
            if "openalex" not in sources:
                sys.exit(1)  # arreter si Scopus est l'unique source

    # 3. Collecte OpenAlex (si selectionnee)
    if "openalex" in sources:
        if not run_step("OpenAlex Data Collection",
                        ["python", "core/openalex_collector.py"]):
            print("[Warn] OpenAlex collection failed.")
            if "scopus" not in sources:
                sys.exit(1)

    # 4. Fusion des resultats
    merge_results(sources)

    # 5. Analyse bibliometrique R
    if not run_step("Bibliometric R Analysis",
```

```
                    ["Rscript", "core/analysis_engine.R"]):
        print("[Warn] R Analysis failed. Continuing...")

    # 6. Classement des articles
    if not run_step("Ranking & Filtering",
                    ["python", "core/paper_ranker.py"]):
        sys.exit(1)

    # 7. Redaction et compilation PDF
    if not run_step("LaTeX Review Generation & PDF Build",
                    ["python", "core/review_drafter.py"]):
        sys.exit(1)

    print("\n" + "="*50)
    print("  MISSION SUCCESSFUL")
    print("="*50)
```

Listing 4 – Corps de main() — Orchestrateur principal

→ OUTPUT — Résultat produit

Aucune valeur de retour. Les effets observables sont :

- Affichage de la progression dans la console
- Création de tous les fichiers intermédiaires et finaux
- Code de sortie 0 (succès) ou 1 (échec) transmis au système d'exploitation

5. Agent 2 — Scopus Collector (core/scopus_collector.py)

5.1. Rôle et positionnement

Scopus Collector — L'Enquêteur Elsevier

Rôle métier	Interroger la base Scopus (la plus grande base bibliographique multi-disciplinaire payante au monde)
Analogie	Un bibliothécaire expert qui connaît parfaitement le langage de requête de la bibliothèque Elsevier et en extrait les meilleures références
Entrée	Thème de recherche (texte libre) + Clé API Scopus
Sortie	`scopus_metadata.csv` + `results.bib`
API utilisée	`https://api.elsevier.com/content/search/scopus`

Concept : Scopus et son API

Scopus est une base de données bibliographique propriétaire d'Elsevier qui index plus de 90 millions d'articles. L'accès se fait via une *API REST* : le programme envoie une requête HTTP (comme un navigateur web), et le serveur Scopus répond avec des données JSON (format structuré lisible par machine).

5.2. Fonction 2.1 — get_config()

SIGNATURE COMPLÈTE — `get_config()` → `dict`

Charge et retourne la configuration depuis le fichier JSON de paramétrage.

INPUTS — Données d'entrée

Aucun paramètre. La fonction cherche directement le fichier `agent_config.json` dans le répertoire courant.

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
CONFIG_FILE = "agent_config.json"   # constante globale

def get_config():
    # os.path.exists = "est-ce que ce fichier existe ?"
    if os.path.exists(CONFIG_FILE):
        with open(CONFIG_FILE, 'r', encoding='utf-8') as f:
            # json.load convertit le JSON en dictionnaire Python
            return json.load(f)
    return {}  # retourner un dictionnaire vide si fichier
```

```
        absent
```

Listing 5 – Corps de get-config()

OUTPUT — Résultat produit

- `dict` : Dictionnaire Python contenant tous les paramètres (ex : `{"gemini_api_key": "...", "theme_description": "..."}`)
- `{}` : Dictionnaire vide si le fichier est introuvable

5.3. Fonction 2.2 — `suggest_boolean_query()`

SIGNATURE COMPLÈTE — `suggest_boolean_query(client, model_id: str, theme: str) → str`

Utilise l'IA Gemini pour générer une requête Scopus booléenne optimale.

INPUTS — Données d'entrée

Paramètre	Type	Description
`client`	`genai.Client`	Instance du client Gemini (objet de connexion à l'API Google)
`model_id`	`str`	Identifiant du modèle Gemini à utiliser (ex : `"gemini-3.1-flash-lite-preview"`)
`theme`	`str`	Le thème de recherche en texte libre (ex : `"équité intergénérationnelle et fiscalité carbone"`)

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def suggest_boolean_query(client, model_id, theme):
    print(f"[Collector] Consulting AI for theme: '{theme}'...")

    # Construction du prompt (instruction pour l'IA)
    prompt = f"""
    You are a bibliometric expert. Generate a professional Scopus
    boolean search query for the theme: "{theme}".

    Requirements:
    1. Use TITLE-ABS-KEY(...) format.
    2. Include synonyms and related academic terms.
    3. Use logical operators (AND, OR, wildcard *).
```

```
    4. Ensure the query is highly targeted to high-impact research.
    5. Return ONLY the string of the query itself.

    Example: TITLE-ABS-KEY("artificial intelligence" AND "healthcare")
    """

    try:
        # Appel a l'API Gemini
        response = client.models.generate_content(
            model=model_id,
            contents=prompt
        )
        # Recuperer le texte genere et nettoyer les backticks
        query = response.text.strip().replace("`", "")

        # Validation : s'assurer que la requete est au bon format
        if "TITLE-ABS-KEY" not in query:
            query = f"TITLE-ABS-KEY({query})"

        return query

    except Exception as e:
        # En cas d'echec, utiliser une requete basique de secours
        print(f"[Warn] AI query failed: {e}. Falling back.")
        return f'TITLE-ABS-KEY("{theme}")'
```

Listing 6 – Corps de suggest_boolean-query()

Qu'est-ce qu'une requête booléenne Scopus ?

Analogie : comprendre facilement

Une requête booléenne, c'est comme parler à un moteur de recherche très sophistiqué avec un langage précis. Par exemple :

— **AND** : les deux mots doivent être présents

— **OR** : l'un ou l'autre (ou les deux)

— ***** (wildcard) : remplace n'importe quelle fin de mot (`decarboni*` trouve "decarbonization", "decarbonisation", etc.)

— **TITLE-ABS-KEY** : chercher dans le titre, le résumé ET les mots-clés de l'article

Exemple de requête générée par l'IA : `TITLE-ABS-KEY("intergenerational equity" AND ("carbon tax" OR "decarboni*"))`

OUTPUT — Résultat produit

— `str` : Requête Scopus booléenne prête à l'emploi (ex : `TITLE-ABS-KEY("fiscal policy" AND "intergenerational" OR "equity")`)

— Si l'IA échoue : requête de repli basique (`TITLE-ABS-KEY("thème brut")`)

5.4. Fonction 2.3 — `fetch_scopus_data()`

SIGNATURE COMPLÈTE — `fetch_scopus_data(query: str, api_key: str, limit: int=50) → list`

Interroge l'API Scopus et retourne les entrées brutes d'articles.

INPUTS — Données d'entrée

Paramètre	Type	Description
`query`	`str`	La requête Scopus booléenne générée par `suggest_boolean_query()`
`api_key`	`str`	La clé d'authentification Scopus (Elsevier)
`limit`	`int`	Nombre maximum d'articles à récupérer (défaut : 50, max Scopus : 25/appel)

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```
def fetch_scopus_data(query, api_key, limit=50):
    print(f"[Collector] Querying Scopus: {query[:100]}...")

    url = "https://api.elsevier.com/content/search/scopus"
```

```python

    # En-tetes HTTP : authentification + format de reponse
    headers = {
        "X-ELS-APIKey": api_key,     # cle d'acces Elsevier
        "Accept": "application/json" # demander une reponse JSON
    }

    # Parametres de la requete HTTP
    params = {
        "query": query,
        "count": min(limit, 25),  # Scopus limite a 25/appel
        "field": "eid,dc:title,dc:creator,prism:publicationName,"
                 "prism:coverDate,prism:doi,citedby-count,"
                 "dc:description,authkeywords,affiliation-name"
    }

    try:
        # Envoyer la requete HTTP GET avec timeout de 20 secondes
        resp = requests.get(url, headers=headers,
                            params=params, timeout=20)

        if resp.status_code == 200:   # 200 = succes HTTP
            data = resp.json()
            # Extraire la liste d'articles de la reponse
            entries = data.get("search-results", {}).get("entry", [])

            # Verifier s'il y a une erreur applicative dans la reponse
            if entries and "error" in entries[0]:
                print(f"[Warn] Scopus API Error: {entries[0]['error']}")
                return []

            return entries          # retourner les articles trouves

        else:
            print(f"[Error] HTTP {resp.status_code}: {resp.text[:200]}")

    except Exception as e:
```

```
        print(f"[Error] Connection failed: {e}")

    return []  # Retourner liste vide en cas d'echec
```

Listing 7 – Corps de fetch_scopus-data()

Concepts expliqués :

Concept : API REST et HTTP

Une *API REST* fonctionne comme passer une commande au restaurant : vous envoyez une requête (commande) avec vos spécifications (paramètres), le serveur prépare et retourne la réponse (plat). Le code **200** signifie "succès" — tout comme un serveur qui dit "voilà votre commande". Un code **404** ou **403** signifie "non trouvé" ou "accès refusé".

Concept : Le champ `field` de Scopus

Ce paramètre dit à Scopus exactement *quelles informations* inclure dans la réponse. Sans le spécifier, Scopus renvoie des dizaines de champs inutiles. En le précisant, on optimise la vitesse et la taille de la réponse.

→ OUTPUT — Résultat produit

- `list[dict]` : Liste de dictionnaires, chaque dictionnaire représentant un article avec ses métadonnées brutes Scopus
- `[]` : Liste vide si l'appel échoue ou si aucun article trouvé

Exemple d'une entrée de la liste retournée :

```
{
  "dc:title": "Intergenerational equity and carbon taxation",
  "dc:creator": "Smith J.",
  "prism:coverDate": "2023-06-15",
  "citedby-count": "47",
  "prism:doi": "10.1016/j.ecolecon.2023.107612",
  "dc:description": "This paper examines..."
}
```

5.5. Fonction 2.4 — `process_entries()`

SIGNATURE COMPLÈTE — `process_entries(entries: list) → pd.DataFrame`

Transforme la liste brute de dictionnaires Scopus en un DataFrame pandas structuré.

INPUTS — Données d'entrée

`entries`	`list[dict]`	Liste brute retournée par `fetch_scopus_data()`

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def process_entries(entries):
    if not entries:              # si la liste est vide
        return pd.DataFrame() # retourner un tableau vide

    # Convertir la liste de dictionnaires en tableau (DataFrame)
    df = pd.DataFrame(entries)

    # Renommer la colonne de citations pour standardisation
    if 'citedby-count' in df.columns:
        df = df.rename(columns={'citedby-count': 'citations'})
    elif 'citations' not in df.columns:
        df['citations'] = 0    # creer la colonne si absente

    return df
```

Listing 8 – Corps de process-entries()

Concept : DataFrame pandas

Un *DataFrame* est l'équivalent Python d'un tableau Excel — des lignes et des colonnes. La bibliothèque `pandas` est l'outil standard en Python pour manipuler des données tabulaires. `pd.DataFrame(entries)` convertit automatiquement une liste de dictionnaires en tableau : les clés deviennent les noms de colonnes, les valeurs deviennent les cellules.

OUTPUT — Résultat produit

- `pd.DataFrame` : Tableau avec autant de lignes que d'articles et les colonnes suivantes : `dc:title`, `dc:creator`, `prism:coverDate`, `citations`, `prism:doi`, `dc:description`, `authkeywords`
- `pd.DataFrame()` vide si la liste d'entrée est vide

5.6. Fonction 2.5 — `export_to_bibtex()` (version Scopus)

SIGNATURE COMPLÈTE — `export_to_bibtex(df: pd.DataFrame, filename: str="results.bib") → None`

Convertit le DataFrame Scopus en fichier BibTeX standard pour les logiciels de gestion bibliographique.

INPUTS — Données d'entrée

`df`	`pd.DataFrame`	Tableau de métadonnées (sortie de `process_entries()`)
`filename`	`str`	Nom du fichier de sortie (défaut : `"results.bib"`)

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def export_to_bibtex(df, filename="results.bib"):
    if df.empty:     # si le tableau est vide
        with open(filename, "w", encoding="utf-8") as f:
            f.write("")    # creer un fichier vide
        return

    lines = []
    for i, row in df.iterrows():  # boucle sur chaque ligne du tableau
        ref_id = f"id{i+1}"        # cle de citation ex: id1, id2...

        # Extraire et nettoyer chaque champ
        title = str(row.get("dc:title","No Title")).replace("{","").replace("}","")
        author = str(row.get("dc:creator", "Anonymous"))
        year   = str(row.get("prism:coverDate", "2024"))[:4]  # premiere 4 chars
        doi    = str(row.get("prism:doi", ""))
        journal = str(row.get("prism:publicationName", "Journal"))
        abstract = str(row.get("dc:description", ""))
        cit    = str(row.get("citations", 0))
        keywords = str(row.get("authkeywords", "")).replace("|", "; ")

        # Construire l'entree BibTeX
        bib  = f"@article{{{ref_id},\n"
        bib += f"  author = {{{author}}},\n"
        bib += f"  title = {{{title}}},\n"
```

```
        bib += f"  journal = {{{journal}}},\n"
        bib += f"  year = {{{year}}},\n"
        bib += f"  doi = {{{doi}}},\n"
        bib += f"  abstract = {{{abstract}}},\n"
        bib += f"  keywords = {{{keywords}}},\n"
        bib += f"  TC = {{{cit}}},\n"
        bib += f"  note = {{Citations: {cit}}}\n"
        bib += "}\n"
        lines.append(bib)

    # Ecrire tout le contenu dans le fichier
    with open(filename, "w", encoding="utf-8") as f:
        f.write("\n".join(lines))
    print(f"[Collector] BibTeX saved to {filename}")
```

Listing 9 – Corps de export_to-bibtex() — version Scopus

Qu'est-ce que le format BibTeX ?

Analogie : comprendre facilement

BibTeX est le format standard pour stocker des références bibliographiques. C'est comme une fiche d'identité standardisée pour chaque article, que tous les logiciels académiques (LaTeX, Zotero, Mendeley, Word) savent lire et afficher correctement.

Exemple de sortie :

```
@article{id1,
  author = {Smith, J.},
  title = {Intergenerational equity and carbon tax},
  journal = {Ecological Economics},
  year = {2023},
  doi = {10.1016/j.ecolecon.2023.107612},
  TC = {47},
}
```

→ OUTPUT — Résultat produit

Aucune valeur de retour (`None`). Effet : création/écrasement du fichier BibTeX sur le disque.

6. Agent 3 — OpenAlex Collector (`core/openalex_collector.py`)

6.1. Rôle et positionnement

OpenAlex Collector — L'Enquêteur Open Access

Rôle métier	Interroger OpenAlex, la plus grande base de données scientifiques en accès libre au monde
Analogie	Un deuxième enquêteur qui fouille dans une bibliothèque publique mondiale (250+ millions d'articles, entièrement gratuite)
Différence Scopus	OpenAlex couvre plus d'articles mais avec moins de métadonnées. Les deux sources se complètent
Entrée	Thème + clé API OpenAlex (optionnelle)
Sortie	`openalex_metadata.csv` + `openalex_results.bib`

6.2. Fonction 3.1 — `reconstruct_abstract()`

C'est la fonction la plus originale de cet agent. OpenAlex ne stocke pas les résumés en texte brut mais dans un format compressé appelé *index inversé*.

SIGNATURE COMPLÈTE — `reconstruct_abstract(inverted_index: dict) → str`

Reconstruit un résumé d'article lisible à partir du format index inversé d'OpenAlex.

INPUTS — Données d'entrée

`inverted_index` `dict | None`	Dictionnaire où chaque clé est un mot et la valeur est la liste de ses positions dans le texte.

Exemple de format reçu :

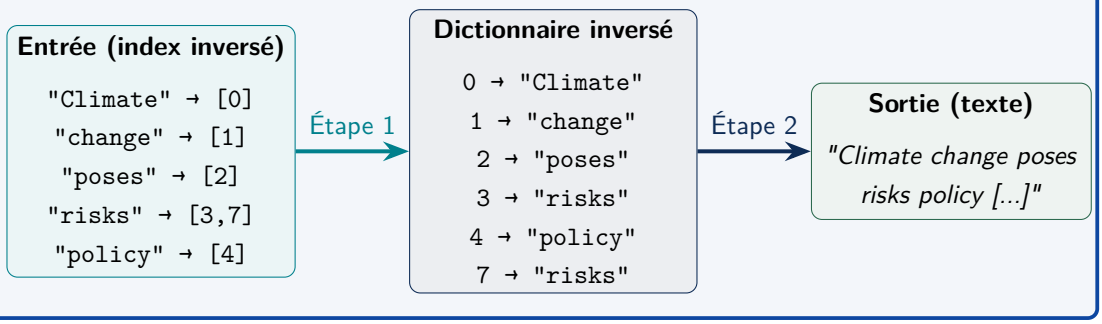
```
{
  "Climate":  [0],
  "change":   [1],
  "poses":    [2],
  "significant": [3],
  "risks":    [4],
  "Climate":  [0],   // <- meme mot peut apparaitre a plusieurs positions
  "policy":   [5, 12]  // "policy" apparait en position 5 et 12
}
```

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def reconstruct_abstract(inverted_index):
    if not inverted_index:    # si None ou vide
        return ""

    try:
        word_index = {}   # dictionnaire inverse : position -> mot

        # Inverser le dictionnaire : mot -> [pos] devient pos -> mot
        for word, locations in inverted_index.items():
            for loc in locations:
                word_index[loc] = word   # associer chaque position a son mot

        # Trouver la position la plus haute = longueur du texte
        highest_index = max(word_index.keys())

        # Reconstruire le texte dans l'ordre
        abstract_parts = []
        for i in range(highest_index + 1):
            # Recuperer le mot a la position i (ou "" si manquant)
            abstract_parts.append(word_index.get(i, ""))

        # Rejoindre les mots avec un espace
        return " ".join(abstract_parts)

    except Exception:
        return ""   # en cas d'erreur, retourner chaine vide
```

Listing 10 – Corps de reconstruct-abstract()

Illustration du processus de reconstruction :



→ OUTPUT — Résultat produit

— `str` : Le résumé complet reconstruit en texte lisible

— `""` : Chaîne vide si l'index est absent, vide ou corrompu

6.3. Fonction 3.2 — `suggest_search_query()` (version OpenAlex)

SIGNATURE COMPLÈTE — `suggest_search_query(client, model_id: str, theme: str) → str`

Génère une requête optimisée pour OpenAlex (syntaxe différente de Scopus).

INPUTS — Données d'entrée

Identiques à `suggest_boolean_query()` de Scopus, mais le prompt demande une requête au format OpenAlex (strings simples avec AND/OR, sans TITLE-ABS-KEY).

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def suggest_search_query(client, model_id, theme):
    prompt = f"""
    You are a bibliometric expert. Generate a professional search
    query (string) for OpenAlex for the theme: "{theme}".
    OpenAlex uses simple strings with "AND" "OR" and quotes.

    Requirements:
    1. Focus on keywords, synonyms, and related academic terms.
    2. Use quotes for exact phrases.
    3. Return ONLY the string of the query itself.

    Example: "artificial intelligence" AND "healthcare"
    """

    try:
        response = client.models.generate_content(
            model=model_id,
            contents=prompt
        )
        query = response.text.strip().replace("`", "")
        return query
    except Exception as e:
        print(f"[Warn] AI query generation failed: {e}.")
        return theme  # repli : utiliser le theme brut
```

Listing 11 – Corps de suggest_search-query() pour OpenAlex

Différence Scopus vs OpenAlex pour les requêtes :

	Scopus	OpenAlex
Format	`TITLE-ABS-KEY(...)`	Chaîne simple
Opérateurs	AND, OR, * (wildcard)	AND, OR, guillemets
Exemple	`TITLE-ABS-KEY("equity" AND "tax*")`	`"equity" AND "taxation"`

→ OUTPUT — Résultat produit

`str` : Requête optimisée pour l'API OpenAlex.

6.4. Fonction 3.3 — `fetch_openalex_data()`

SIGNATURE COMPLÈTE — `fetch_openalex_data(query: str, api_key: str, limit: int=50) → list`

Interroge l'API OpenAlex et retourne les résultats bruts.

INPUTS — Données d'entrée

`query`	`str`	Requête de recherche OpenAlex
`api_key`	`str`	Clé API optionnelle (accès prioritaire au "Polite Pool")
`limit`	`int`	Nombre max d'articles (pris depuis la config)

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def fetch_openalex_data(query, api_key, limit=50):
    url = "https://api.openalex.org/works"

    headers = {"Accept": "application/json"}
    if api_key:
        headers["api-key"] = api_key  # acces "Polite Pool" (prioritaire)

    params = {
        "search": query,
        "per_page": min(limit, 100),  # OpenAlex max: 200/appel
        "mailto": "bilel.ammouri@example.com"
        # Le champ mailto identifie poliment le client
        # et donne acces au "Polite Pool" (serveurs rapides)
    }

    try:
```

```
        resp = requests.get(url, headers=headers,
                            params=params, timeout=30)
        if resp.status_code == 200:
            data = resp.json()
            return data.get("results", []) # liste d'articles
        else:
            print(f"[Error] OpenAlex HTTP {resp.status_code}")
    except Exception as e:
        print(f"[Error] Connection failed: {e}")

    return []
```

Listing 12 – Corps de fetch_openalex-data()

Concept : Le "Polite Pool" d'OpenAlex

OpenAlex offre deux "files" d'accès : la file standard (ralentie en cas de surcharge) et le *Polite Pool* (serveurs dédiés, accès prioritaire). Pour accéder au Polite Pool, il suffit d'envoyer son adresse email dans le paramètre `mailto` ou dans le header. C'est entièrement gratuit et uniquement utilisé à des fins de contact en cas de problème.

OUTPUT — Résultat produit

`list[dict]` : Liste de dictionnaires au format OpenAlex (structure différente de Scopus — nécessite la normalisation de `process_results()`).

6.5. Fonction 3.4 — `process_results()`

SIGNATURE COMPLÈTE — `process_results(results: list) → pd.DataFrame`

Normalise les résultats OpenAlex au format compatible avec Scopus (colonnes identiques).

INPUTS — Données d'entrée

`results`	`list[dict]`	Résultats bruts de l'API OpenAlex

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```
def process_results(results):
    if not results:
        return pd.DataFrame()
```

```python

    processed = []
    for item in results:
        # Extraction des champs OpenAlex
        title = item.get("display_name", "No Title")
        year  = item.get("publication_year", 2024)
        citations = item.get("cited_by_count", 0)
        doi   = item.get("ids", {}).get("doi", "")

        # Authors : liste d'objets -> string "A; B; C"
        authors_list = [
            a.get("author", {}).get("display_name", "")
            for a in item.get("authorships", [])
        ]
        author = "; ".join(authors_list) if authors_list else "Anonymous"

        # Journal (primary location)
        source = item.get("primary_location", {}).get("source")
        journal = source.get("display_name", "Unknown") if source else "Unknown"

        # Reconstruction du resume depuis l'index inverse
        abstract = reconstruct_abstract(
            item.get("abstract_inverted_index")
        )

        # Normalisation : renommer les colonnes au format Scopus
        processed.append({
            "dc:title": title,
            "dc:creator": author,
            "prism:coverDate": str(year),
            "citations": citations,
            "prism:doi": doi,
            "prism:publicationName": journal,
            "dc:description": abstract,
            "source": "OpenAlex"      # marqueur de provenance
        })

    return pd.DataFrame(processed)
```

Listing 13 – Corps de process-results() — normalisation OpenAlex

Pourquoi normaliser les colonnes ? La fusion des données Scopus et OpenAlex dans

`merge_results()` nécessite que les deux DataFrames aient exactement les mêmes noms de colonnes. En renommant les champs OpenAlex au format de Scopus (`dc:title`, `dc:creator`, etc.), le code de fusion peut traiter les deux sources de façon identique.

→ OUTPUT — Résultat produit

`pd.DataFrame` : Tableau normalisé au format Scopus avec colonnes standardisées. La colonne `"source"` vaut `"OpenAlex"` pour traçabilité.

7. Agent 4 — Analysis Engine R (core/analysis_engine.R)

7.1. Rôle et positionnement

Analysis Engine — L'Analyste Scientométrique

Rôle métier	Analyser la qualité et l'influence des articles collectés via des métriques bibliométriques avancées
Langage	R (langage statistique) + package Bibliometrix (standard académique mondial)
Analogie	Un expert en réputation académique qui calcule le "poids" de chaque article dans la communauté scientifique
Entrée	`results.bib` (bibliographie fusionnée)
Sorties	`citation_metrics.csv`, `bibliometric_summary.png`

Concept : Bibliometrix

Bibliometrix est le package R de référence mondial pour l'analyse bibliométrique. Il est utilisé par des milliers de chercheurs universitaires pour cartographier des champs scientifiques, identifier les auteurs influents et visualiser l'évolution des connaissances.

7.2. Analyse du script R complet

```r
# Configurer le chemin de bibliotheques locales
local_libs <- file.path(getwd(), "..", "R_libs")
dir.create(local_libs, showWarnings = FALSE)
.libPaths(c(local_libs, .libPaths()))   # ajouter en tete du chemin

# Installer Bibliometrix si absent (mode binaire Windows)
safe_install <- function(pkg) {
  if (!require(pkg, character.only = TRUE, quietly = TRUE)) {
    install.packages(pkg,
      repos = "https://cloud.r-project.org",
      lib = local_libs,
      type = "win.binary"   # binaire pre-compile, sans compilation C++
    )
  }
}

tryCatch({
  safe_install("bibliometrix")
  library(bibliometrix)
}, error = function(e) {
  cat("[R Error] Bibliometrix installation failed.\n")
})
```

```r

# Verifier l'existence et la taille du fichier BibTeX
if (file.exists("results.bib") && file.size("results.bib") > 0) {

    # 1. Charger et convertir le BibTeX en DataFrame R
    M <- tryCatch({
        convert2df(file = "results.bib",
                   dbsource = "scopus",
                   format = "bibtex")
    }, error = function(e) { NULL })

    if (!is.null(M) && nrow(M) > 0) {

        # 2. Calculer les metriques bibliometriques
        res <- biblioAnalysis(M)

        # 3. Generer les visualisations
        try({
            png("bibliometric_summary.png",
                width=1200, height=1000, res=150)
            plot(x = res, k = 10, pause = FALSE)
            dev.off()
        })

        # 4. Extraire les metriques cles par article
        ranking <- M[, c("TI","AU","PY","TC","DI"), drop=FALSE]
        # TI = titre, AU = auteur, PY = annee,
        # TC = total citations, DI = DOI
        write.csv(ranking, "citation_metrics.csv", row.names=FALSE
            )
    }
}
```

Listing 14 – Script R

Métriques produites par `biblioAnalysis()` :

Champ	Nom complet	Signification simple
TC	**Total Citations**	Combien de fois cet article a été cité par d'autres
TI	**Title**	Titre de l'article
AU	**Authors**	Liste des auteurs
PY	**Publication Year**	Année de publication
DI	**DOI**	Identifiant unique universel

8. ★ Agent 5 — Paper Ranker (core/paper_ranker.py)

8.1. Rôle et positionnement

★ Paper Ranker — Le Jury de Sélection

Rôle métier	Classer tous les articles collectés et sélectionner le Top 15 pour la synthèse
Analogie	Un jury d'experts qui note chaque candidature selon des critères objectifs et publie le classement final
Entrée	`collected_metadata.csv` + `citation_metrics.csv` (optionnel)
Sortie	`ranked_articles.json` — Top 15 sélectionné

8.2. Fonction unique — rank_papers()

SIGNATURE COMPLÈTE — `rank_papers()` → `None`

Orchestrateur du classement : chargement, enrichissement, scoring, sélection et export du Top 15.

INPUTS — Données d'entrée

Aucun paramètre direct. La fonction lit directement les fichiers du disque. Par ordre de priorité :

1. `collected_metadata.csv` (fusionné)
2. `scopus_metadata.csv` (repli)
3. `openalex_metadata.csv` (dernier repli)

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def rank_papers():
    # 1. Charger le fichier de metadonnees (avec replis)
    metadata_file = "collected_metadata.csv"
    if not os.path.exists(metadata_file):
        if os.path.exists("scopus_metadata.csv"):
            metadata_file = "scopus_metadata.csv"
        elif os.path.exists("openalex_metadata.csv"):
            metadata_file = "openalex_metadata.csv"
        else:
            print("[Error] No metadata file found.")
            return

    df_scopus = pd.read_csv(metadata_file)

```

```python
    if df_scopus.empty:
        with open("ranked_articles.json", "w") as f:
            json.dump([], f)
        return

    # 2. Normaliser la colonne citations
    cit_col = None
    if 'citations' in df_scopus.columns:
        cit_col = 'citations'
    elif 'citedby-count' in df_scopus.columns:
        cit_col = 'citedby-count'

    if cit_col is None:
        df_scopus['citations_count'] = 0  # pas de donnees de citations
    else:
        df_scopus['citations_count'] = pd.to_numeric(
            df_scopus[cit_col], errors='coerce'
        ).fillna(0)
        # errors='coerce' : valeurs non-numeriques = NaN
        # fillna(0) : remplacer NaN par 0

    # 3. Enrichir avec les metriques R si disponibles
    if os.path.exists("citation_metrics.csv"):
        df_r = pd.read_csv("citation_metrics.csv")

        # Cle de jointure : titre en minuscules, sans espaces superflus
        df_scopus['clean_title'] = df_scopus['dc:title'].astype(str) \
                                    .str.lower().str.strip()
        df_r['clean_title']      = df_r['TI'].astype(str) \
                                   .str.lower().str.strip()

        # Fusion LEFT JOIN sur le titre nettoye
        df_merged = pd.merge(
            df_scopus,
            df_r[['clean_title', 'TC']],
            on='clean_title',
            how='left'   # garder tous les articles meme sans match R
        )
        # Preferer les citations R (plus precises) si disponibles
        df_merged['citations_count'] = df_merged['TC'].fillna(
```

```
            df_merged['citations_count']
        )
    else:
        df_merged = df_scopus

    # 4. Calculer le score de classement
    df_merged['year_val'] = pd.to_numeric(
        df_merged['prism:coverDate'].astype(str).str[:4],
        errors='coerce'
    ).fillna(2000)

    # FORMULE : Score = (Citations x 2) + (Annee - 2000)
    df_merged['rank_score'] = (
        df_merged['citations_count'] * 2
    ) + (
        df_merged['year_val'] - 2000
    )

    # 5. Trier et selectionner le Top 15
    df_final = df_merged.sort_values(by='rank_score',
        ascending=False)
    top_n = df_final.head(15)

    # 6. Construire la liste de sortie JSON
    result = []
    for _, row in top_n.iterrows():
        result.append({
            "title":    row.get("dc:title", "Untitled"),
            "authors":  row.get("dc:creator", "Anonymous"),
            "year":     int(row.get("year_val", 2024)),
            "abstract": row.get("dc:description", ""),
            "journal":  row.get("prism:publicationName", "
                Unknown"),
            "citations":int(row.get("citations_count", 0)),
            "doi":      row.get("prism:doi", "")
        })

    with open("ranked_articles.json", "w", encoding="utf-8")
        as f:
        json.dump(result, f, indent=4)

    print(f"[Ranker] Success: {len(result)} articles
        prioritized.")
```

Listing 15 – Corps complet de rank-papers()

Explication de la formule de scoring :

$$\text{Score} = (\text{Citations} \times 2) + (\text{Année} - 2000)$$

Analogie : comprendre facilement

Intuition de la formule :

— **Citations** $\times$ **2** : Le nombre de fois qu'un article a été cité par d'autres chercheurs est le meilleur indicateur objectif de son *impact scientifique*. On le pondère double car c'est le critère le plus important.

— **(Année** $-$ **2000)** : La récence d'un article compte car les connaissances évoluent. Un article de 2024 ajoute 24 points, un article de 2010 ajoute 10 points.

Exemple :

Article	Citations	Année	Score
Article A	200	2018	$200 \times 2 + 18 = \mathbf{418}$
Article B	50	2024	$50 \times 2 + 24 = \mathbf{124}$
Article C	10	2024	$10 \times 2 + 24 = \mathbf{44}$

OUTPUT — Résultat produit

Aucune valeur de retour (`None`). Effet : création de `ranked_articles.json`.

Exemple de sortie JSON :

```
[
  {
    "title": "Intergenerational equity and decarbonization",
    "authors": "Smith J.; Jones A.",
    "year": 2022,
    "abstract": "This paper examines...",
    "journal": "Ecological Economics",
    "citations": 142,
    "doi": "10.1016/j.ecolecon.2022.107612"
  },
  ...  // 14 autres articles dans le meme format
]
```

9. Agent 6 — Review Drafter (core/review_drafter.py)

9.1. Rôle et positionnement

Review Drafter — Le Rédacteur Académique IA

Rôle métier	Rédiger automatiquement une revue de littérature académique complète et compiler le PDF final
Analogie	Un auteur académique expert doté d'une mémoire parfaite des 15 meilleurs articles, capable de rédiger une synthèse de 3000 mots en 10 minutes
Entrée	`ranked_articles.json` + `agent_config.json`
Sortie	`main.tex` + `main.pdf`

9.2. Fonction 6.1 — load_config() et load_data()

SIGNATURE COMPLÈTE — `load_config() → dict` | `load_data() → list`

Chargement des paramètres de configuration et des données d'articles classés.

INPUTS — Données d'entrée

Aucun paramètre — lecture directe des fichiers du disque.

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def load_config():
    if not os.path.exists("agent_config.json"):
        return {}
    with open("agent_config.json", "r", encoding="utf-8") as f:
        return json.load(f)

def load_data():
    # Source prioritaire : fichier JSON pre-classe
    if os.path.exists("ranked_articles.json"):
        with open("ranked_articles.json", "r", encoding="utf-8") as f:
            return json.load(f)

    # Repli : si le classement n'existe pas, utiliser le CSV brut
    if os.path.exists("scopus_metadata.csv"):
        import pandas as pd
        df = pd.read_csv("scopus_metadata.csv")
```

```
        return df.to_dict('records')  # convertir en liste de dicts

    return []  # aucune donnee disponible
```

Listing 16 – Fonctions de chargement

OUTPUT — Résultat produit

— `load_config()` → `dict` : Dictionnaire de configuration complet

— `load_data()` → `list[dict]` : Liste des articles (jusqu'à 15 entrées)

9.3. Fonction 6.2 — `latex_escape()`

SIGNATURE COMPLÈTE — `latex_escape(text: str) → str`

Nettoie le texte généré par l'IA pour le rendre compatible avec la compilation LaTeX.

INPUTS — Données d'entrée

`text`	`str`	Texte brut généré par Gemini (peut contenir des caractères spéciaux LaTeX)

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```
def latex_escape(text):
    # Table de correspondance : caractere -> sequence LaTeX
    chars = {
        "&":  r"\&",     # esperluette -> \&
        "%":  r"\%",     # pourcentage -> \%
        "$":  r"\$",     # dollar -> \$
        "#":  r"\#",     # diese -> \#
        "_":  r"\_",     # underscore -> \_
        "{":  r"\{",     # accolade ouverte -> \{
        "}":  r"\}",     # accolade fermee -> \}
        "~":  r"\textasciitilde{}",
        "^":  r"\^{}",
    }
    for k, v in chars.items():
        text = text.replace(k, v)
    return text
```

Listing 17 – Corps de latex-escape()

Pourquoi cette étape est critique ? En LaTeX, certains caractères ont une signification spéciale dans le code de composition. Par exemple, % commence un commentaire et $

active le mode mathématique. Si l'IA génère du texte contenant ces caractères dans leur sens littéral (ex : "la TVA est de 20%"), LaTeX interprétera le reste de la ligne comme un commentaire et le rapport sera corrompu. Cette fonction préfixe chaque caractère dangereux avec un backslash (\) pour le "neutraliser".

OUTPUT — Résultat produit

`str` : Texte sécurisé pour la compilation LaTeX (tous les caractères spéciaux sont échappés).

9.4. Fonction 6.3 — `initialize_client()`

SIGNATURE COMPLÈTE — `initialize_client(config: dict) → genai.Client | None`

Initialise la connexion à l'API Gemini de Google.

INPUTS — Données d'entrée

`config`	`dict`	Dictionnaire de configuration (doit contenir `gemini_api_key`)

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def initialize_client(config):
    api_key = config.get("gemini_api_key")

    # Validation : rejeter les cles placeholder
    if not api_key or api_key == "YOUR_GEMINI_API_KEY_HERE":
        print("[Error] Gemini API Key is missing.")
        return None

    try:
        client = genai.Client(api_key=api_key)
        return client
    except Exception as e:
        print(f"[Error] Failed to initialize Gemini Client: {e}")
        return None
```

Listing 18 – Corps de initialize-client()

OUTPUT — Résultat produit

- `genai.Client` : Objet de connexion Gemini prêt à envoyer des requêtes
- `None` : Si la clé est invalide ou manquante

9.5. Fonction 6.4 — `call_with_retry()`

SIGNATURE COMPLÈTE — `call_with_retry(client, model_id: str, prompt: str, config=None, max_retries: int=5) → str | None`

Appelle l'API Gemini avec gestion automatique des limites de débit (rate limiting).

INPUTS — Données d'entrée

`client`	`genai.Client`	Instance de connexion Gemini
`model_id`	`str`	Identifiant du modèle Gemini
`prompt`	`str`	Instruction complète à envoyer à l'IA
`config`	`GenerateContentConfig`	Paramètres de génération (température, etc.)
`max_retries`	`int`	Nombre maximum de tentatives (défaut : 5)

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def call_with_retry(client, model_id, prompt, config=None, max_retries=5):
    for attempt in range(max_retries):   # 0, 1, 2, 3, 4
        try:
            response = client.models.generate_content(
                model=model_id,
                contents=prompt,
                config=config
            )
            return response.text.strip()  # succes : retourner le texte

        except Exception as e:
            err_msg = str(e)

            # Detecter l'erreur "trop de requetes" (429)
            if "RESOURCE_EXHAUSTED" in err_msg or "429" in err_msg:
                # Delai exponentiel + aleatoire pour eviter les collisions
                wait_time = (2 ** attempt) + random.random() + 5
                # attempt=0 -> 2^0 + rnd + 5 = ~6s
```

```
19                  # attempt=1 -> 2^1 + rnd + 5 = ~7s
20                  # attempt=2 -> 2^2 + rnd + 5 = ~9s
21                  # attempt=3 -> 2^3 + rnd + 5 = ~13s
22                  # attempt=4 -> 2^4 + rnd + 5 = ~21s
23                  print(f"[Wait] Rate limit. Retrying in {
                        wait_time:.1f}s...")
24                  time.sleep(wait_time)
25              else:
26                  # Erreur non-recuperable : logs et sortie
27                  print(f"[Error] API Call failed: {err_msg}")
28                  return None
29
30      return None  # echec apres toutes les tentatives
```

Listing 19 – Corps de call_with-retry() — Exponential Backoff

Qu'est-ce que l'Exponential Backoff ?

Analogie : comprendre facilement

Imaginez que vous essayez d'appeler quelqu'un qui a son téléphone occupé. Plutôt que de rappeler immédiatement en boucle (ce qui surcharge la ligne), vous attendez : 6 secondes, puis 7, puis 9, puis 13, puis 21. Cette stratégie d'attente croissante s'appelle *Exponential Backoff* — elle est standard dans les systèmes professionnels pour gérer les surcharges d'API.
La formule est : **temps d'attente** $= 2^{\text{tentative}} + \text{aléatoire} + 5$ secondes.

OUTPUT — Résultat produit

— `str` : Texte généré par Gemini si au moins une tentative réussit

— `None` : Si toutes les tentatives échouent

9.6. Fonction 6.5 — `generate_dynamic_structure()`

SIGNATURE COMPLÈTE — `generate_dynamic_structure(client, model_id: str, theme: str, articles: list) → list[str]`

Demande à Gemini de proposer une structure thématique logique pour la revue de littérature.

INPUTS — Données d'entrée

client	genai.Client	Connexion Gemini
model_id	str	Modèle Gemini
theme	str	Thème de recherche
articles	list	Liste des 15 meilleurs articles (titres + années)

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def generate_dynamic_structure(client, model_id, theme, articles):
    # Construire la liste des titres pour le contexte
    article_list = ""
    for i, a in enumerate(articles[:15]):
        title = a.get('dc:title', a.get('title', 'Unknown'))
        year  = str(a.get('prism:coverDate', a.get('year', '2025')))[:4]
        article_list += f"[{i+1}] {title} ({year})\n"

    # Prompt demandant la structure en JSON
    prompt = f"""
    You are a senior academic researcher writing a Q1 journal review.
    Theme: "{theme}"
    Top articles:
    {article_list}

    Propose a logical, thematic structure for this review.
    Return the result as a JSON array of 4-6 section titles.
    Return ONLY THE JSON ARRAY. No explanation.
    """

    res = call_with_retry(client, model_id, prompt)
    if not res:
        # Repli : structure par defaut si Gemini echoue
        return ["Theoretical Framework",
                "Methodological Innovations",
                "Global Policy Impacts",
                "Future Frontiers"]

    try:
        text = res.strip()
        # Nettoyer les blocs de code Markdown si presents
        if "```json" in text:
            text = text.split("```json")[1].split("```")[0].strip()
```

```
        elif "```" in text:
            text = text.split("```")[1].split("```")[0].strip()

        structure = json.loads(text)  # parser le JSON en liste Python
        return structure

    except Exception:
        return ["Theoretical Framework",
                "Methodological Innovations",
                "Global Policy Impacts",
                "Future Frontiers"]
```

Listing 20 – Corps de generate_dynamic-structure()

→ OUTPUT — Résultat produit

- `list[str]` : Liste de 4 à 6 titres de sections thématiques proposés par l'IA (ex : `["Fiscal Instruments for Decarbonization", "Intergenerational Justice Frameworks", ...]`)
- Si Gemini échoue : structure de secours générique à 4 sections

9.7. Fonction 6.6 — `draft_section()`

C'est la fonction cœur du système — celle qui produit le contenu académique réel.

SIGNATURE COMPLÈTE — `draft_section(client, model_id: str, theme: str, section_title: str, articles: list, target_words: int) → str`

Rédige le contenu d'une section complète de la revue de littérature.

INPUTS — Données d'entrée

`client`	`genai.Client`	Connexion Gemini
`model_id`	`str`	Modèle à utiliser
`theme`	`str`	Thème global de la recherche
`section_title`	`str`	Titre de la section à rédiger
`articles`	`list`	Liste des articles avec leurs résumés
`target_words`	`int`	Nombre de mots cibles (ex : 700)

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def draft_section(client, model_id, theme, section_title,
                  articles, target_words):
    print(f"[Drafter] Drafting section: {section_title}...")

    # Construire le contexte des articles (limiter a 1200 chars/resume)
    context = ""
    for i, a in enumerate(articles[:15]):
        title    = a.get('dc:title', a.get('title', 'Unknown'))
        author   = a.get('dc:creator', a.get('authors', 'Anonymous'))
        year     = str(a.get('prism:coverDate', a.get('year', '2025')))[:4]
        abstract_raw = a.get('dc:description', a.get('abstract', ''))
        abstract = str(abstract_raw) if pd.notnull(abstract_raw) \
                   else "No abstract available."

        context += f"ARTICLE {i+1}: Title: {title}\n"
        context += f"Authors: {author}\nYear: {year}\n"
        context += f"Abstract: {abstract[:1200]}...\n\n"

    # Prompt de redaction academique
    prompt = f"""
    You are writing a Q1 journal literature review in English.
    Theme: "{theme}"
    Section: "{section_title}" (~{target_words} words)

    Articles Data:
    {context}

    Instructions:
    1. Write a dense academic synthesis with critical analysis.
    2. Focus STRICTLY on how articles relate to: "{theme}".
    3. Compare different perspectives in the provided articles.
    4. You MUST use LaTeX citation format \citep{{idX}}
       (X = index 1 to {min(15, len(articles))}).
    5. Maintain a formal, doctoral-level tone in academic English.

```

```
    Return ONLY the LaTeX body text (no preamble, no section header).
    """

    # Basse temperature = texte plus factuel, moins "creatif"
    config = types.GenerateContentConfig(temperature=0.3)

    res = call_with_retry(client, model_id, prompt, config=config)
    return res if res else f"Error generating content for {section_title}."
```

Listing 21 – Corps de draft-section()

Le rôle de la température dans la génération de texte :

Concept : Température dans les LLMs

La *température* est un paramètre qui contrôle le degré de créativité/aléatoire dans la génération de texte par l'IA.

- **Température = 0.0** : Réponse déterministe, toujours la même, très factuelle
- **Température = 0.3** : Légèrement créatif tout en restant précis — idéal pour la rédaction académique
- **Température = 1.0** : Plus créatif et varié — mieux pour la fiction
- **Température = 2.0** : Très aléatoire, parfois incohérent

Ici, 0.3 est choisi car la revue de littérature doit être précise et cohérente, mais pas robotique.

→ OUTPUT — Résultat produit

`str` : Corps LaTeX de la section (texte académique structuré avec citations `\citep{idX}`, prêt à être inséré dans le document principal). En cas d'échec : message d'erreur informatif.

9.8. Fonction 6.7 — `create_latex_master()`

SIGNATURE COMPLÈTE — `create_latex_master(config: dict, content: str) → None`

Assemble le document LaTeX complet avec préambule, contenu et bibliographie.

INPUTS — Données d'entrée

`config`	`dict`	Configuration (titre, auteur, mots-clés)
`content`	`str`	Corps du document généré (toutes sections assemblées)

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

La fonction génère le préambule LaTeX complet avec :

- `\documentclass[11pt,a4paper]{article}`
- Packages : `inputenc`, `natbib`, `hyperref`, `xcolor`, `geometry`
- Page de titre automatique (`\maketitle`)
- Environnement `abstract` avec résumé générique
- Corps : `content` inséré directement
- Bibliographie : `\bibliographystyle{apalike}` + `\bibliography{results}`

Le document final est écrit dans `main.tex`.

OUTPUT — Résultat produit

Création/écrasement de `main.tex` sur le disque.

9.9. Fonction 6.8 — `run_compilation()`

SIGNATURE COMPLÈTE — `run_compilation()` → `None`

Compile le fichier LaTeX en PDF via pdflatex et bibtex (3 passes nécessaires).

INPUTS — Données d'entrée

Aucun paramètre. Utilise `main.tex` et `results.bib` présents sur le disque.

CORPS DE LA FONCTION — Ce qui se passe à l'intérieur

```python
def run_compilation():
    try:
        devnull = open(os.devnull, 'w')    # supprimer les logs verbeux

        # 1ere passe : generer les fichiers auxiliaires (.aux, .toc)
        subprocess.run(["pdflatex", "-interaction=nonstopmode", "main.tex"],
                       check=False, stdout=devnull)

```

```
        # 2eme passe : traiter la bibliographie BibTeX
        subprocess.run(["bibtex", "main"],
                       check=False, stdout=devnull)

        # 3eme passe : resoudre les references croisees
        subprocess.run(["pdflatex", "-interaction=nonstopmode", "main.tex"],
                       check=False, stdout=devnull)

        # 4eme passe : finaliser (certains cas necessitent 4 passes)
        subprocess.run(["pdflatex", "-interaction=nonstopmode", "main.tex"],
                       check=False, stdout=devnull)

        print("[Success] PDF complete: main.pdf")
    except Exception as e:
        print(f"[Warn] PDF build error: {e}")
```

Listing 22 – Corps de run-compilation()

Pourquoi 3 passes LaTeX ?

Analogie : comprendre facilement

LaTeX fonctionne comme imprimer un livre avec des renvois de pages. La première passe "imprime" tout mais dit "voir page ?" pour les références. La deuxième passe (bibtex) résout les citations. Les passes suivantes mettent à jour les numéros de pages une fois toutes les citations résolues. C'est inhérent au fonctionnement de LaTeX, et c'est pourquoi 3 à 4 compilations sont nécessaires.

→ OUTPUT — Résultat produit

Création de `main.pdf` dans le répertoire racine du projet.

10. ⚙ Configuration Complète du Système

10.1. Le fichier `agent_config.json` — Tableau de Bord Unique

```
{
    "author_name":      "Bilel Ammouri",
    "project_title":    "Intergenerational Equity in Decarbonization and Fiscal Policy",
    "scopus_api_key":   "ade7466450720df2422d3b3275ddc2a6",
    "openalex_api_key": "9UILMlDW5oaodN5t5lDEVh",
    "article_count":    50,
    "intro_word_count": 200,
    "review_word_count": 3000,
    "theme_description": "Comment concevoir des politiques de decarbonation
                          et des structures fiscales qui garantissent
                          l'equite intergenerationnelle ?",
    "gemini_api_key":   "AIzaSy...",
    "model_name":       "gemini-3.1-flash-lite-preview"
}
```

Listing 23 – Fichier agent-config.json actuel du projet

Paramètre	Type	Agent concerné	Impact
`author_name`	string	review_drafter	Page de titre du rapport
`project_title`	string	Tous	Titre principal
`theme_description`	string	Collecteurs + Drafter	Requête de recherche
`scopus_api_key`	string	scopus_collector	Authentification Elsevier
`openalex_api_key`	string	openalex_collector	Accès Polite Pool
`article_count`	integer	Collecteurs	Taille de la collecte
`intro_word_count`	integer	review_drafter	Longueur de l'intro
`review_word_count`	integer	review_drafter	Longueur totale
`gemini_api_key`	string	Collecteurs + Drafter	Authentification Google
`model_name`	string	Collecteurs + Drafter	Modèle IA utilisé

10.2. Modifier le thème de recherche

Pour lancer une nouvelle recherche sur un sujet différent, seules 2 lignes sont à modifier :

```
{
  "project_title": "VOTRE NOUVEAU SUJET ICI",
  "theme_description": "La question de recherche detaillee ici..."
      ,
  // Toute la suite reste identique
}
```

11. Approche Pédagogique : Du Simple au Complexe

11.1. Les trois niveaux de compréhension

Niveau 1 — Compréhension universelle

ARA est un robot assistant qui lit des articles scientifiques à votre place.
Vous lui demandez un sujet, il cherche dans des milliers de livres, choisit les meilleurs, et vous écrit un résumé propre et structuré.

Niveau 2 — Vision décisionnelle

ARA est un système d'automatisation de la recherche documentaire et de la synthèse bibliographique. Il transforme un thème de recherche brut en un rapport académique structuré de 3000+ mots, fondé sur les 15 sources les plus influentes identifiées parmi 50+ articles collectés depuis Scopus et OpenAlex. Le temps de traitement passe de 3 semaines à 15 minutes.

Niveau 3 — Vision architecturale

Pipeline multi-agents Python orchestré par un Master Agent via `subprocess`. Collecte parallèle depuis Scopus (REST API Elsevier, requête TITLE-ABS-KEY générée par LLM) et OpenAlex (API gratuite, reconstruction d'index inversé). Fusion et déduplication par DOI/titre (pandas concat + drop_duplicates). Scoring bibliométrique R/Bibliometrix + scoring Python pondéré ($2\times$citations + ancienneté). Rédaction LLM (Gemini API, température 0.3, exponential backoff). Compilation LaTeX automatique (pdflatex $\times 3$ + bibtex).

11.2. Tableau comparatif des analogies utilisées

Concept Technique	Analogie Simple	Exemple Concret
API REST	Téléphone à une bibliothèque	Appeler Scopus pour demander des articles
DataFrame pandans	Tableau Excel Python	Tableau avec lignes = articles, colonnes = infos
Requête booléenne	Langage de bibliothécaire	TITLE-ABS-KEY("equity" AND "carbon")
Index inversé	Puzzle numéroté	Reconstituer un texte depuis ses positions de mots
Exponential Backoff	Rappel avec patience	Réessayer 6s, 7s, 9s, 13s si ligne occupée
Température LLM	Créativité de l'écrivain	0.3 = factuel, 1.0 = créatif, 2.0 = aléatoire

Concept Technique	Analogie Simple	Exemple Concret
Déduplication	Supprimer les doublons d'une liste	Un article présent sur Scopus ET OpenAlex = 1 seule entrée
BibTeX	Fiche d'identité d'un article	Format standard lu par tous les logiciels bibliographiques
Subprocess	Commander depuis PowerShell	Python lance Rscript comme si vous le tapiez
JSON	Formulaire structuré	`{"clé": "valeur"}` — lisible par machine et humain

12. Tableau de Bord Complet des Fonctions

Fonction	Agent	Input	Output
`find_r_executable()`	Master	—	str | None
`run_step()`	Master	name, command	bool
`merge_results()`	Master	sources : list	None (fichiers)
`main()`	Master	—	None
`get_config()`	Scopus	—	dict
`suggest_boolean_query()`	Scopus	client, model, theme	str
`generate_boolean_query()`	Scopus	theme	str (repli)
`fetch_scopus_data()`	Scopus	query, api_key, limit	list[dict]
`process_entries()`	Scopus	entries : list	DataFrame
`export_to_bibtex()`	Scopus	df, filename	None (fichier)
`suggest_search_query()`	OpenAlex	client, model, theme	str
`reconstruct_abstract()`	OpenAlex	inverted_index : dict	str
`fetch_openalex_data()`	OpenAlex	query, api_key, limit	list[dict]
`process_results()`	OpenAlex	results : list	DataFrame
`export_to_bibtex()`	OpenAlex	df, filename	None (fichier)
`rank_papers()`	Ranker	— (lecture fichiers)	None (JSON)
`load_config()`	Drafter	—	dict
`load_data()`	Drafter	—	list
`latex_escape()`	Drafter	text : str	str
`initialize_client()`	Drafter	config : dict	Client | None
`call_with_retry()`	Drafter	client, model, prompt	str | None
`generate_dynamic_structure()`	Drafter	client, model, theme, articles	list[str]
`draft_section()`	Drafter	client, model, theme, section, articles, words	str
`create_latex_master()`	Drafter	config, content	None (fichier)
`run_compilation()`	Drafter	—	None (PDF)

13. Résilience et Gestion des Erreurs

13.1. Stratégie de repli (Fallback Strategy)

ARA est conçu pour ne jamais planter brutalement. À chaque étape critique, un mécanisme de repli est prévu.

Scénario d'erreur	Comportement	Mécanisme
Gemini IA indisponible	Requête de repli simple	Thème brut utilisé comme requête
Scopus API erreur	Warning + continue	Si OpenAlex disponible, pipeline continue
OpenAlex API erreur	Warning + continue	Si Scopus disponible, pipeline continue
R non installé	Warning + continue	Classement sans métriques Bibliometrix
Résumé OpenAlex manquant	Chaîne vide	`reconstruct_abstract()` retourne ""
Rate limit Gemini (429)	Retry x5 avec exponential backoff	Délais croissants automatiques
Fichier de données manquant	Repli sur source alternative	Ordre : collected → scopus → openalex
LaTeX compilation échoue	Warning sans crash	Le .tex est conservé pour compilation manuelle
JSON malformé depuis Gemini	Structure par défaut 4 sections	Try/except dans parse_structure()

13.2. Flux de résilience visualisé

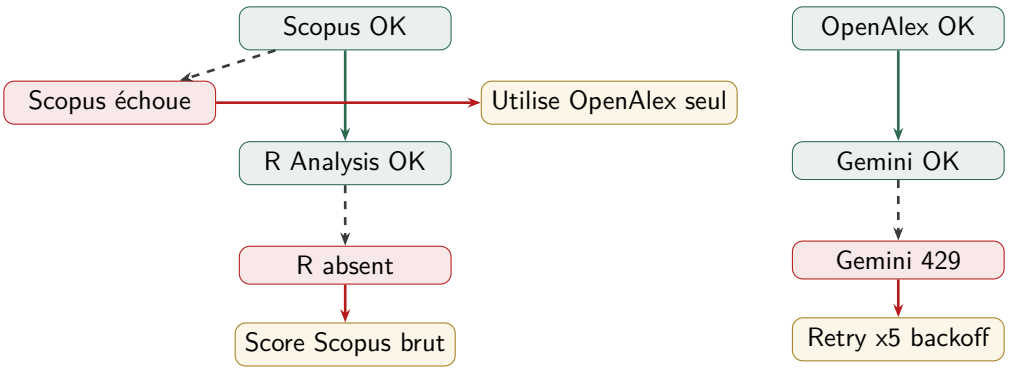


FIGURE 3 – Stratégie de repli du système ARA à chaque point de défaillance possible

14. ▶ Guide d'Utilisation Pratique

14.1. Installation et prérequis

Composant	Version minimale	Rôle dans le système
Python	3.10+	Langage principal (Master, Collecteurs, Ranker, Drafter)
R	4.2+	Analyse bibliométrique (Analysis Engine)
LaTeX (MiKTeX)	Dernière version	Compilation du rapport PDF final
Scopus API Key	—	Accès à la base Elsevier
Gemini API Key	—	Intelligence artificielle Google

14.2. Installation des dépendances Python

```
# Depuis le dossier racine du projet
pip install -r requirements.txt

# requirements.txt contient :
# pandas
# requests
# google-genai
```

Listing 24 – Installation des dépendances Python

14.3. Lancement du pipeline complet

```
# Depuis le dossier racine
python master_agent.py

# Le systeme vous demandera :
# Select source [1-3, default=3]:
#   1 = Scopus uniquement
#   2 = OpenAlex uniquement
#   3 = Les deux (recommande)
```

Listing 25 – Lancement du système ARA

14.4. Lancement d'un agent individuel (débogage)

```
# Tester uniquement le collecteur Scopus
python core/scopus_collector.py

# Tester uniquement le collecteur OpenAlex
python core/openalex_collector.py
```

```python

# Tester uniquement le classement
python core/paper_ranker.py

# Tester uniquement la redaction
python core/review_drafter.py

# Tester l'analyse R
Rscript core/analysis_engine.R
```

Listing 26 – Tests individuels des agents

14.5. Arborescence des fichiers générés

```
Autonomous_Research_Agent V2/
|-- scopus_metadata.csv        <- donnees Scopus brutes
|-- openalex_metadata.csv      <- donnees OpenAlex brutes
|-- collected_metadata.csv     <- donnees fusionnees et
    deduplicatees
|-- results.bib                <- bibliographie BibTeX fusionnee
|-- openalex_results.bib       <- bibliographie OpenAlex d'origine
|-- citation_metrics.csv       <- metriques R (si disponible)
|-- bibliometric_summary.png   <- graphiques bibliometriques
|-- ranked_articles.json       <- Top 15 classes avec scores
|-- main.tex                   <- rapport LaTeX genere
|-- main.pdf                   <- rapport final compile
|-- main.aux, main.log...      <- fichiers auxiliaires LaTeX
```

Listing 27 – Fichiers produits apres une execution complete

15. Conclusion et Recommandations Stratégiques

Ce qu'il faut retenir

Les 6 messages essentiels à retenir :

1. **Un vrai problème d'entreprise résolu** : La veille scientifique est chronophage et coûteuse. ARA l'automatise entièrement, réduisant 3 semaines de travail à 15 minutes.
2. **18 fonctions Python documentées** : Chaque composant est compris, expliqué et modifiable indépendamment.
3. **Architecture résiliente** : Le système ne plante jamais brutalement — chaque point de défaillance a un mécanisme de repli.
4. **Intelligence réelle** : Gemini comprend le sens du thème et génère des requêtes et du contenu de qualité académique, pas seulement des mots-clés.
5. **Livrable professionnel** : Un PDF académique de 3000+ mots avec citations LaTeX, prêt à soumettre ou présenter.
6. **Configuration triviale** : Un seul fichier JSON à modifier pour changer de sujet ou d'utilisateur.

15.1. Pistes d'évolution recommandées

Feuille de Route Technique Recommandée

- ☑ **Nouvelles sources** : Ajouter PubMed (sciences de la vie), ArXiv (physique/IA), SSRN (économie) comme agents collecteurs supplémentaires
- ☑ **Interface Web** : Développer un portail web pour lancer des recherches sans ligne de commande (Flask ou Streamlit)
- ☑ **Multilinguisme** : Configurer le Drafter pour générer en français, arabe, espagnol selon la audience cible
- ☑ **Veilles automatiques** : Planifier des exécutions hebdomadaires ou mensuelles via Windows Task Scheduler ou Cron
- ☑ **Intégration BI** : Connecter les sorties JSON/CSV aux outils de tableau de bord (Power BI, Tableau) pour visualisation continue
- ☑ **Base de données** : Stocker l'historique des recherches dans SQLite ou PostgreSQL pour des analyses comparatives
- ☑ **Parallélisation** : Lancer Scopus et OpenAlex simultanément (threading Python) pour réduire le temps de collecte de 50%

ARA — La recherche académique à la vitesse de l'intelligence artificielle

6 agents · 18 fonctions · 15 minutes · 1 rapport professionnel

---

Document généré par le système Antigravity. Bilel Ammouri — Avril 2025. Version 2.0 — Documentation étendue avec analyse complète des fonctions Python. Ce document est destiné à un usage interne et confidentiel. Toute reproduction est soumise à autorisation.